SE3308 Algorithm Design I — Assignment 4

Exercises 6.5, 6.10, 6.17, 6.20, 6.21, and 6.22

Xilang Chen 524031910668

Arithmetic, comparisons, Boolean operations, and table accesses take constant time. Coin values and subset-sum inputs are positive integers. All ties between optimal choices may be broken arbitrarily.

Exercise 6.5: Pebbling a checkerboard

(a) Legal patterns. Number the rows $1, 2, 3, 4$ from top to bottom. A pattern is a subset of these rows containing no consecutive pair. There are exactly eight:

$$\mathcal{P} = \big\{\varnothing, \{1\}, \{2\}, \{3\}, \{4\}, \{1,3\}, \{1,4\}, \{2,4\}\big\}.$$

There is one empty pattern, four singleton patterns, and three two-row patterns. Three pebbles cannot fit in a column without vertical adjacency: each of the row pairs $\{1,2\}$ and $\{3,4\}$ contains at most one pebble. This also proves that every legal placement uses at most $2n$ pebbles, so the supply of $2n$ imposes no additional restriction.

(b) States and recurrence. Patterns $P, Q \in \mathcal{P}$ are compatible exactly when $P \cap Q = \varnothing$, because equal row indices in neighboring columns would create horizontal adjacency. Diagonal adjacency imposes no restriction. For the integer $a_{r,j}$ in row r, column j, define

$$w_j(P) = \sum_{r \in P} a_{r,j}.$$

Let $F(j, P)$ be the maximum weight of a legal placement in columns $1, \ldots, j$ whose last column has pattern P. Initialize

$$F(0, \varnothing) = 0, \qquad F(0, P) = -\infty \quad (P \neq \varnothing).$$

Then, for $j = 1, \ldots, n$,

$$\boxed{F(j, P) = w_j(P) + \max_{\substack{Q \in \mathcal{P} \\ Q \cap P = \varnothing}} F(j-1, Q).}$$

The optimal value is $\max_{P \in \mathcal{P}} F(n, P)$. The empty pattern must be retained: entries may be negative, and placing no pebbles is legal.

Recovering the placement. For every (j, P), store a maximizing predecessor $B(j, P)$. Choose P_n maximizing $F(n, P_n)$, then set $P_{j-1} = B(j, P_j)$ for $j = n, n-1, \ldots, 1$. Place a pebble at (r, j) exactly when $r \in P_j$.

Exercise 6.5 (continued)

Algorithm 1 Optimal checkerboard placement

function PEBBLE($a[1..4, 1..n]$)
  $\mathcal{P} \leftarrow \{\varnothing, \{1\}, \{2\}, \{3\}, \{4\}, \{1,3\}, \{1,4\}, \{2,4\}\}$
  $F(0, \varnothing) \leftarrow 0$; $F(0, P) \leftarrow -\infty$ for $P \neq \varnothing$
  for $j = 1, \ldots, n$ **do**
    for $P \in \mathcal{P}$ **do**
      $Q^* \leftarrow \arg\max_{Q \in \mathcal{P}:\, Q \cap P = \varnothing} F(j-1, Q)$
      $F(j, P) \leftarrow F(j-1, Q^*) + \sum_{r \in P} a_{r,j}$
      $B(j, P) \leftarrow Q^*$
    end for
  end for
  $P \leftarrow \arg\max_{Q \in \mathcal{P}} F(n, Q)$; $z \leftarrow F(n, P)$
  $S \leftarrow \varnothing$
  for $j = n, n-1, \ldots, 1$ **do**
    Add every (r, j) with $r \in P$ to S
    $P \leftarrow B(j, P)$
  end for
  return (z, S)
end function

Correctness. Proceed by induction on j. At $j = 0$, the empty placement has weight zero and is the only feasible state.

Consider any legal placement in the first j columns ending in P. Deleting its last column leaves a legal placement in the first $j-1$ columns ending in some compatible Q. By the induction hypothesis, its prefix weight is at most $F(j-1, Q)$, so its total weight is at most the stated recurrence.

Conversely, take an optimal prefix for any compatible Q with finite value and append P. Pattern P has no vertical conflict, and $P \cap Q = \varnothing$ prevents every new horizontal conflict. No earlier column is adjacent to column j. Thus this constructs a legal placement of weight $F(j-1, Q) + w_j(P)$, including the maximizing choice. The recurrence therefore gives exactly the optimum for every state.

Every complete placement has one of the eight final patterns, so the final maximum is globally optimal. The stored predecessors attain the recurrence at each step; following them reconstructs a legal placement with that value.

Complexity. There are $8n$ noninitial states, each examining at most eight predecessors. Pattern weights and compatibility tests take constant time because there are only four rows. Filling the tables and reconstructing the placement therefore take $\boxed{O(n)}$ time. The tables and output use $O(n)$ space. If only the optimal value is wanted, two layers of eight values suffice, giving $O(1)$ auxiliary space.

Exercise 6.10: Counting heads

Let $H(i, h)$ be the probability of exactly h heads among the first i coins. Use

$$H(0,0) = 1, \qquad H(i,h) = 0 \quad \text{if } h < 0 \text{ or } h > i.$$

Conditioning on the outcome of coin i gives

$$\boxed{H(i,h) = (1-p_i)H(i-1,h) + p_i H(i-1,h-1).}$$

Return $H(n,k)$; if $k < 0$ or $k > n$, return zero immediately.

Algorithm. Only entries $0, \ldots, k$ are needed: the number of heads cannot decrease. Store them in an array q, and update from high indices to low indices so both terms still refer to the preceding layer.

Algorithm 2 Probability of exactly k heads

```
1:  function CountingHeads(p[1..n], k)
2:      if k < 0 or k > n then
3:          return 0
4:      end if
5:      q[0] ← 1; q[1..k] ← 0
6:      for i = 1, ..., n do
7:          for h = min(i, k), ..., 1 in decreasing order do
8:              q[h] ← (1 − p_i)q[h] + p_i q[h − 1]
9:          end for
10:         q[0] ← (1 − p_i)q[0]
11:     end for
12:     return q[k]
13: end function
```

Correctness. The event of exactly h heads in the first i tosses is the disjoint union of:

- exactly h heads in the first $i-1$ tosses, followed by a tail;
- exactly $h-1$ heads in the first $i-1$ tosses, followed by a head.

Independence makes their probabilities $(1-p_i)H(i-1,h)$ and $p_i H(i-1,h-1)$, respectively. Their sum is the recurrence. Starting from the certain empty outcome, induction on i proves all computed probabilities correct.

In the in-place implementation, when updating $q[h]$, both $q[h]$ and $q[h-1]$ retain their previous-layer values. The update of $q[0]$ is performed last. Thus $q[h] = H(i,h)$ after iteration i for every stored index, including the zero entries with $h > i$, and the returned value is correct.

Complexity and boundaries. Initialization costs $O(k+1)$ and each coin costs $O(k+1)$, giving $O(n(k+1)) \subseteq \boxed{O(n^2)}$ time for $n \geq 1$ and $0 \leq k \leq n$. Space is $O(k+1)$. For $n = 0$, the answer is 1 exactly when $k = 0$. The recurrence also handles $p_i = 0$ or 1 without division.

Exercise 6.17: Change-making with unlimited coins

Let $x_1, \ldots, x_n$ be positive integer denominations and let $v \geq 0$. Define $R(s)$ to mean that amount s can be made using any number of coins of the available denominations. The empty collection gives $R(0) = \mathsf{true}$. For $s = 1, \ldots, v$, use

$$\boxed{R(s) = \bigvee_{\substack{1 \leq i \leq n \\ x_i \leq s}} R(s - x_i),}$$

where an OR over no terms is false. Evaluate amounts in increasing order and return $R(v)$.

Algorithm 3 Unlimited-coin change-making

```
 1: function CanMakeChange(x[1..n], v)
 2:     if v = 0 then return true
 3:     if n = 0 then return false
 4:     R[0] ← true; R[1..v] ← false
 5:     for s = 1, ..., v do
 6:         for i = 1, ..., n do
 7:             if x_i ≤ s and R[s − x_i] then
 8:                 R[s] ← true; last[s] ← i
 9:                 break
10:             end if
11:         end for
12:     end for
13:     return R[v]
14: end function
```

Correctness. We prove the state definition by strong induction on s. Amount zero is attainable by using no coins.

If a term $R(s - x_i)$ is true, the induction hypothesis gives coins totaling $s - x_i$. Adding one coin of denomination x_i makes amount s, so every true transition is feasible. Reusing x_i is allowed because the supply is unlimited.

Conversely, any representation of a positive amount s contains at least one coin, say of denomination x_i. Remove that coin. The remaining total is $s - x_i < s$, and the induction hypothesis makes $R(s - x_i)$ true. The recurrence examines this denomination, so it sets $R(s)$ to true. Therefore $R(v)$ answers the question exactly.

Optional reconstruction. When $R(v)$ is true, start at $s = v$. Output a coin of denomination $x_{\text{last}[s]}$ and replace s by $s - x_{\text{last}[s]}$ until $s = 0$. Every predecessor is reachable, and the amount strictly decreases, so this process terminates and the output sums to v.

Complexity. For $n, v \geq 1$, the v amounts each examine at most n denominations, giving $\boxed{O(nv)}$ time and $O(v + 1)$ space. Reconstruction takes at most v steps, since each denomination is at least one. Zero target and no-denomination cases are handled directly.

The bound is *pseudo-polynomial*: it depends on the numeric value v, whose binary encoding has only $O(\log(v + 1))$ bits.

Exercise 6.20: Optimal binary search trees

Let the sorted keys be $w_1 < \cdots < w_n$, with frequencies $p_1, \ldots, p_n$. Only successful searches for these keys are part of the objective. Count one comparison at the root. For a tree T, the cost is

$$\operatorname{cost}(T) = \sum_{q=1}^{n} p_q\big(\operatorname{depth}_T(w_q) + 1\big),$$

where the root has depth zero.

State. Let $C(i, j)$ be the minimum weighted search cost of a BST containing exactly $w_i, \ldots, w_j$, with comparisons counted from the root of this subtree. Keep the original frequencies; do not renormalize them. An empty interval costs zero:

$$C(i, i-1) = 0 \qquad (1 \le i \le n+1).$$

Compute prefix sums $A(0) = 0$, $A(j) = \sum_{q=1}^{j} p_q$, so that

$$W(i, j) = \sum_{q=i}^{j} p_q = A(j) - A(i-1)$$

can be found in constant time.

Recurrence. If w_r is the root, the BST property forces the left interval to be $[i, r-1]$ and the right interval to be $[r+1, j]$. Every search in $[i, j]$ first compares with w_r, contributing $W(i, j)$. The remaining comparisons occur in the appropriate subtree. Therefore

$$\boxed{C(i, j) = W(i, j) + \min_{r=i,\ldots,j}\big(C(i, r-1) + C(r+1, j)\big).}$$

In particular, $C(i, i) = p_i$. Fill intervals in increasing order of length and record a minimizing root $K(i, j)$.

Correctness. Use induction on interval length. Empty intervals have no comparisons. For a nonempty interval, every valid BST has some root w_r, and its left and right subtrees contain the two forced intervals above. By induction, their weighted costs are at least $C(i, r-1)$ and $C(r+1, j)$. The root adds exactly $W(i, j)$, so no tree costs less than the recurrence's minimum.

Conversely, for any candidate root r, attach optimal trees for the two smaller intervals. The key order guarantees a valid BST, and its cost is exactly $W(i, j) + C(i, r-1) + C(r+1, j)$. Choosing a minimizing root attains the recurrence. Thus $C(1, n)$ is optimal.

Returning a tree. Recursively make $w_{K(i,j)}$ the root of interval $[i, j]$ and rebuild its two subintervals. An empty interval returns a null pointer. This returns an actual optimal tree, rather than only its cost.

Exercise 6.20 (continued)

Algorithm 4 Optimal BST and its reconstruction

```
 1: function OptimalBST(w[1..n], p[1..n])
 2:     Compute prefix sums A[0..n], with A[0] = 0
 3:     C(i, i − 1) ← 0 for every i = 1, ..., n + 1
 4:     for ℓ = 1, ..., n do
 5:         for i = 1, ..., n − ℓ + 1 do
 6:             j ← i + ℓ − 1; C(i, j) ← +∞
 7:             for r = i, ..., j do
 8:                 c ← C(i, r − 1) + C(r + 1, j) + A[j] − A[i − 1]
 9:                 if c < C(i, j) then
10:                     C(i, j) ← c; K(i, j) ← r
11:                 end if
12:             end for
13:         end for
14:     end for
15:     return (C(1, n), Build(1, n))
16: end function
17: function Build(i, j)
18:     if i > j then
19:         return null
20:     end if
21:     r ← K(i, j)
22:     L ← Build(i, r − 1); R ← Build(r + 1, j)
23:     return Node(w_r, L, R)
24: end function
```

Complexity. There are $n(n+1)/2 = O(n^2)$ nonempty intervals. The total number of root candidates examined is

$$\sum_{\ell=1}^{n}(n-\ell+1)\ell = \frac{n(n+1)(n+2)}{6} = O(n^3).$$

Each candidate costs $O(1)$ after the $O(n)$ prefix-sum computation. Thus the total is $\boxed{O(n^3)}$ time and $O(n^2)$ space. Reconstruction creates each of the n nodes once and costs $O(n)$ time. For $n = 0$, $C(1, 0) = 0$ and Build returns the empty tree.

Check on the textbook example. For the ordered words

$$(\texttt{begin}, \texttt{do}, \texttt{else}, \texttt{end}, \texttt{if}, \texttt{then}, \texttt{while})$$

with frequencies $(.05, .40, .08, .04, .10, .10, .23)$, the reconstructed tree can have comparison counts $(2, 1, 4, 5, 3, 4, 2)$ for the respective words. Its root is `do`, and its expected number of comparisons is

$$.40 + 2(.05 + .23) + 3(.10) + 4(.08 + .10) + 5(.04) = \boxed{2.18}.$$

Exercise 6.21: Minimum vertex cover of a tree

Root the input tree at an arbitrary vertex ρ. For each vertex u, let T_u be its rooted subtree. Define $\mathrm{IN}(u)$ as the minimum cover size for edges within T_u with u included, and $\mathrm{OUT}(u)$ as the minimum cover size with u excluded. The edge to u's parent is not part of this subproblem. For a leaf, $\mathrm{IN}(u) = 1$ and $\mathrm{OUT}(u) = 0$. In general,

$$\boxed{\begin{aligned} \mathrm{IN}(u) &= 1 + \sum_{c \in \mathrm{children}(u)} \min\{\mathrm{IN}(c), \mathrm{OUT}(c)\}, \\ \mathrm{OUT}(u) &= \sum_{c \in \mathrm{children}(u)} \mathrm{IN}(c). \end{aligned}}$$

Compute children before parents and return $\min\{\mathrm{IN}(\rho), \mathrm{OUT}(\rho)\}$. An empty tree has answer zero.

Algorithm 5 Tree vertex-cover dynamic program

```
1: function CoverDP(u, parent)
2:     IN(u) ← 1; OUT(u) ← 0
3:     for each neighbor c of u with c ≠ parent do
4:         CoverDP(c, u)
5:         IN(u) ← IN(u) + min{IN(c), OUT(c)}
6:         OUT(u) ← OUT(u) + IN(c)
7:     end for
8: end function
9: Call CoverDP(ρ, null) and return min{IN(ρ), OUT(ρ)}
```

Correctness. Induct on subtree size. For a leaf, there are no internal edges, so including it costs one and excluding it costs zero.

If u is included, every edge (u, c) is already covered; each child may independently be included or excluded. The minimum cost for its subtree is $\min\{\mathrm{IN}(c), \mathrm{OUT}(c)\}$. Adding u itself gives $\mathrm{IN}(u)$.

If u is excluded, every child must be included to cover (u, c). The optimal contribution of child c is therefore $\mathrm{IN}(c)$, giving $\mathrm{OUT}(u)$. Distinct child subtrees have no edges between them because the input is a tree. Their optimal solutions can consequently be combined without conflict. These arguments both lower-bound any feasible cover and construct a cover attaining each recurrence. At the root there is no outside constraint, so taking the smaller state gives the global minimum.

Complexity. With adjacency lists, DFS visits every vertex once and examines each edge twice. Each examination does constant work. Since a nonempty tree has $|E| = |V| - 1$, time is $\boxed{O(|V|)}$ and space is $O(|V|)$, including the recursion stack.

If an actual cover is desired, choose the cheaper root state and walk down the tree: an excluded parent forces its children in, while an included parent lets each child take its cheaper state. This takes another $O(|V|)$ time.

Exercise 6.22: Subset sum with each item used at most once

Let $D(i, s)$ mean that some subset of the indexed items $a_1, \ldots, a_i$ sums to exactly s. Equal values at different indices are distinct available items. Initialize

$$D(0, 0) = \mathsf{true}, \qquad D(0, s) = \mathsf{false} \quad (1 \le s \le t).$$

For $i \ge 1$ and $0 \le s \le t$,

$$\boxed{D(i, s) = D(i-1, s) \ \vee \ \big(s \ge a_i \ \wedge \ D(i-1, s-a_i)\big).}$$

When $s < a_i$, the second term is omitted. The first term excludes item i; the second includes it exactly once. The answer is $D(n, t)$.

Algorithm with compressed space. Use one Boolean array and update sums in *decreasing* order.

Algorithm 6 Subset sum

```
1: function SubsetSum(a[1..n], t)
2:     d[0] ← true; d[1..t] ← false
3:     for i = 1, ..., n do
4:         for s = t, t − 1, ..., a_i do
5:             d[s] ← d[s] ∨ d[s − a_i]
6:         end for
7:     end for
8:     return d[t]
9: end function
```

The inner loop is empty when $a_i > t$.

Correctness. Induct on i. With no items, only sum zero is possible. Any subset of the first i items either excludes item i, giving $D(i-1, s)$, or includes it, leaving a subset of the first $i-1$ items with sum $s - a_i$. Conversely, each true case constructs a valid subset of total s. Thus the recurrence exactly characterizes attainability.

Before iteration i, the invariant is $d[s] = D(i-1, s)$. When processing s in decreasing order, both $d[s]$ and $d[s - a_i]$ still have their previous-layer values: $a_i > 0$ implies $s - a_i < s$, which has not yet been updated. Sums below a_i cannot use this item and remain unchanged. The invariant therefore becomes $d[s] = D(i, s)$ after the iteration, proving the output correct.

Complexity. Initialization takes $O(t+1)$ time, and each of n items uses at most t updates. For $n, t \ge 1$, time is $\boxed{O(nt)}$ and auxiliary space is $O(t+1)$. This is pseudo-polynomial in the binary input. The empty subset also correctly handles a target of zero.

Why the direction matters. With one item $a_1 = 3$ and target 6, an increasing loop would first set $d[3] = \mathsf{true}$ and then incorrectly set $d[6] = \mathsf{true}$ using that new value. The decreasing loop prevents this reuse, unlike unlimited-coin change-making in Exercise 6.17.